

Static variable in java

Though the delivery charge is same for all the customer objects, but in this case, the value of delivery charge is maintained for each and every customer object.

So, instead of storing the delivery charge separately for each and every customer object, it would have been better if we could have maintained a single copy of the delivery charge which could be accessible by all the customer objects.

Usually, values of member variables can be accessed only by that object. But here, we need a variable whose value could be shared by all the objects of a class, i.e., the entire class.

In, object oriented programming languages like Java, there is a concept of creating a **static** variable. A static variable belongs to an entire class and not to a specific object.

When a variable is declared as **Static**, then a single copy of the variable is created and shared among all objects at the class level. Memory allocation for such variables happen only once when the class is loaded in the memory. These variables are also known as class-level variables.

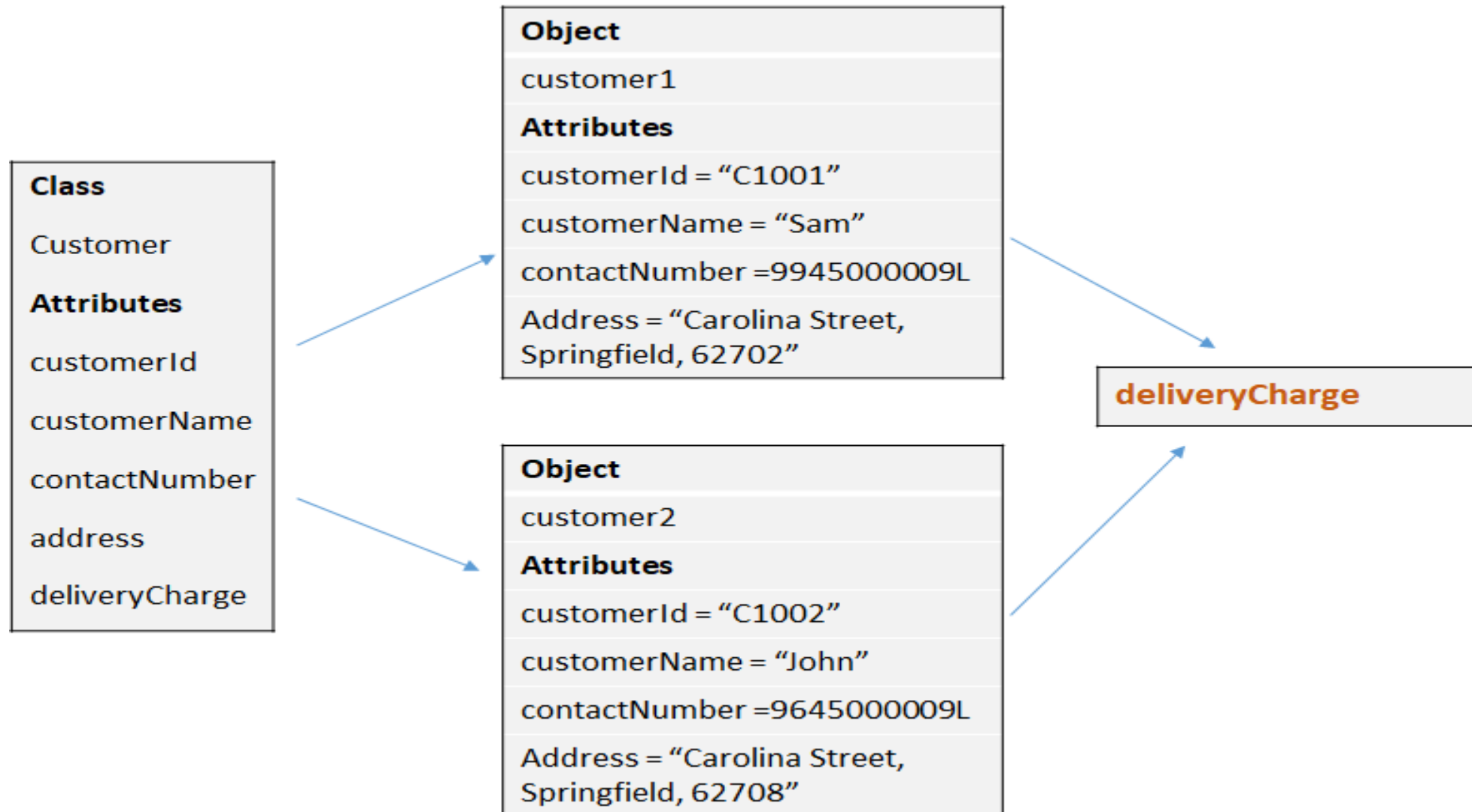
Since static variables and instance variables are both members of the class, they are often referred to as **member variables**.

To create a static member variable, precede its declaration with the keyword static. When a member is declared static, it can be accessed before any object of the class is created and without reference to any object.

Syntax:

```
private static float deliveryCharge; //Declaration of static variable
```

Since the static variable gets created only once, now only one deliveryCharge member variable will be created and all the objects of that class will share that member variable. This can be visualized as



How can we initialize the static variables?

One way to initialize the static variable is to initialize at the time of declaration as shown below

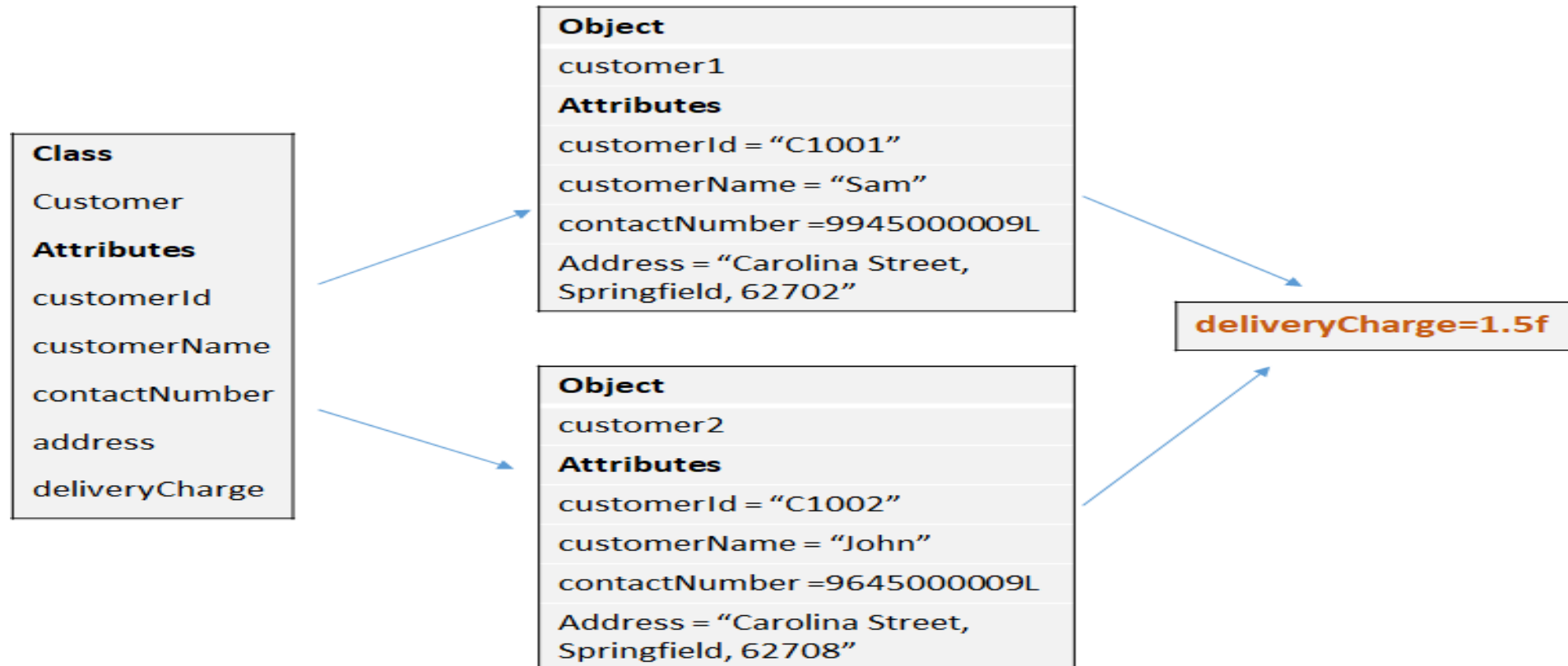
```
private static float deliveryCharge = 1.5f;
```

This is fine if you have to directly initialize a value but this would not work if you need to perform some computation and then initialize the value of a static member variable.

In that case, Java provides one more type of static member called **static block**.

Static blocks are used to initialize static variables when it cannot be done in a single line. They can also be used to add preprocessing if required. In the scenario of the delivery charge, the static variable `deliveryCharge` can be initialized with the help of static block as shown below

```
static {  
    deliveryCharge = 1.5f; // initialize the static variable  
}
```



How can you access the static variable `deliveryCharge` in order to display the total bill amount which has to be paid by the customers?

One way of accessing the static variable is with the help of instances of the class as shown below

```
// Accessing the static variable with the help of instance of the class  
System.out.println("Delivery Charge for customer1 is "+ customer1.deliveryCharge);
```

Since the static variables are not specific to an object, accessing a static variable using an object is not the correct way.

Instead, static variables should be accessed with the help of class name instead of the instance of the class as shown below:

```
// Accessing the static variable with the help of class  
System.out.println("Delivery Charge for customer1 is "+ Customer.deliveryCharge);
```

Next you will see, how can you update or retrieve the value of a static variable?

To update or retrieve the value of a static variable, you need to use the setter and getter methods.

Since static variable is not specific to an object, you need a way to access the setter and getter methods without an object. This is possible by creating **static methods**.

Static methods are methods prefixed with the static keyword. These methods can be accessed without an object of the class. Similar to static variables, they are accessed using the class name.

A static method can be invoked without creating an instance of a class.

A static method can only access static variables and cannot access instance variables.

In case of the delivery charge scenario, you can create static setter and getter methods for deliveryCharge as shown below

```
public static float getDeliveryCharge() {  
    return deliveryCharge;  
}  
public static void setDeliveryCharge(float deliveryCharge) {  
    Customer.deliveryCharge = deliveryCharge;  
}
```

What happens if you try to access the `deliveryCharge` in `setDeliveryCharge()` method with the help of this?

```
public static void setDeliveryCharge(float deliveryCharge) {  
    // Which object's deliveryCharge?  
    this.deliveryCharge = deliveryCharge; // This line will throw error  
}
```

Static blocks and static methods cannot access non-static (instance) members directly since static methods do not belong to any object, so it is not possible to know which object's instance variables should be accessed. Trying to do so will result in a compilation error. However, non-static methods can access static members.